Java Silver（SE 17・1Z0-825-JPN）　3 演算子と制御構造　#07

繰り返し

いつ条件を調べ、どこへ戻り、どこまで抜けるか ― while・do-while・for・拡張 for、break・continue・ラベル

各ページの下に、根拠にした Oracle の資料（JLS 14 章・6.3）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 3（while、do-while、for、拡張 for、break、continue）

1 while と do-while
先に調べるか、後で調べるか。到達できない文はコンパイルエラー

2 for 文
初期化 → 条件 → 本体 → 更新 の順。変数のスコープは for の中

3 拡張 for 文
配列と Iterable だけ。変数は要素のコピー

4 break・continue・ラベル
ラベルなしはいちばん内側。ラベルで外側へ

	条件を調べる時	最低の回数
while	本体の前	0 回
do-while	本体の後	**1 回**
for	本体の前（更新の後）	0 回
拡張 for	要素が残っているか	0 回（空なら）

	ラベルなし	ラベル付き
break	内側の switch・ループを抜ける	ラベルの文を抜ける（if でも可）
continue	内側のループの次の回	ラベルのループの次の回

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JLS 14.12〜14.16

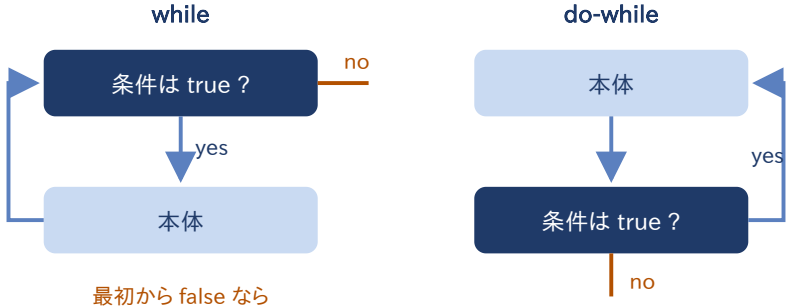
第1章

while 文と do-while 文

先に調べれば 0 回もありうる。後で調べれば必ず 1 回。

前判定と後判定

流れの図（Claude が作成）



最初から false なら
本体は 0 回

while (cond) 文

条件が false でも本体は 1 回

do 文 while (cond);

最後の ; を忘れると ';' expected

i = 5 のとき（Claude が作成）

```
int i = 5;
while (i < 3) {
  System.out.print("w ");   // 0 回
}
do {
  System.out.print("d ");   // 1 回
} while (i < 3);
```

```
d
```

コード（i は 0 から）	結果（JDK 17）
`while (i++ < 3) print(i);`	1 2 3、あとの i は 4
`do print(i); while (++i < 3);`	0 1 2、あとの i は 3

i++ < 3 は増やす前の値で比べる。比べた結果が false の回も i++ は実行されるので、最後に 1 つ多くなる。

出典：JLS 14.12 The while Statement（executes an Expression and a Statement repeatedly until the value of the Expression is false）・14.13 The do Statement（DoStatement: do Statement while (Expression) ;）／結果は JDK 17 で確かめたもの

範囲は { } で決まる ― 無限ループ

字下げにだまされる例（Claude が作成）

```
int i = 0;
while (i < 3)
  System.out.print(i);
  i++;          // ループの外！
```

```
000000000000000000 …（止まらない）
```

コンパイラから見た形

```
while (i < 3) { System.out.print(i); }
i++;            // ここには来ない
```

わざと終わらないループ	抜け方
`while (true) { … }`	中で break・return
`for (;;) { … }`	中で break・return
`do { … } while (true);`	中で break・return

無限ループもコンパイルはできる。i < 3 は定数式ではないので、コンパイラは「いつか false になるかもしれない」と扱う。問題では「無限ループになる」が答えの選択肢になる。

条件の型：boolean か Boolean だけ。`while (1)` や `while (x = 1)`（x は int）はコンパイルエラー（#06 の if と同じ）。

出典：JLS 14.12（WhileStatement: while (Expression) Statement／The Expression must have type boolean or Boolean, or a compile-time error occurs）・14.14.1（If the Expression is not present … the contained Statement is executed）／結果は JDK 17 で確かめたもの

到達できない文はコンパイルエラー

コード	結果（JDK 17）
`while (false) { x++; }`	unreachable statement
`for (int i = 0; false; i++) { }`	unreachable statement
`while (true) { }` のあとの文	unreachable statement
break; の直後の文	unreachable statement
`final boolean F = false; while (F)`	unreachable statement
`if (false) { x++; }`	OK（if は例外）
`boolean b = false; while (b)`	OK（b は定数でない）
`for (int i = 0; i < 0; i++)`	OK（0 回まわるだけ）
`while (true) { if (…) break; }` のあと	OK（break で抜けられる）

判断に使うのは「定数式」だけ。true・false のリテラルや、final で定数に初期化した変数の値を見る。ふつうの変数は値を追いかけない。

if (false) が許される理由：JLS は「デバッグ用の切りかえ」に使えるよう、if だけはわざと到達できるものとして扱うと説明している。

```
static final boolean DEBUG = false;
if (DEBUG) { System.out.println("x = " + x); }  // OK
```

「定数の true の while で、到達できる break がない」ときは、while 文そのものが正常に終わらないので、そのあとの文が到達不能になる。

出典：JLS 14.22 Unreachable Statements（It is a compile-time error if a statement cannot be executed because it is unreachable／The contained statement is reachable iff the while statement is reachable and the condition expression is not a constant expression whose value is false／if (false) { x=3; } does not result in a compile-time error）／JLS 15.29 Constant Expressions／結果は JDK 17 で確かめたもの

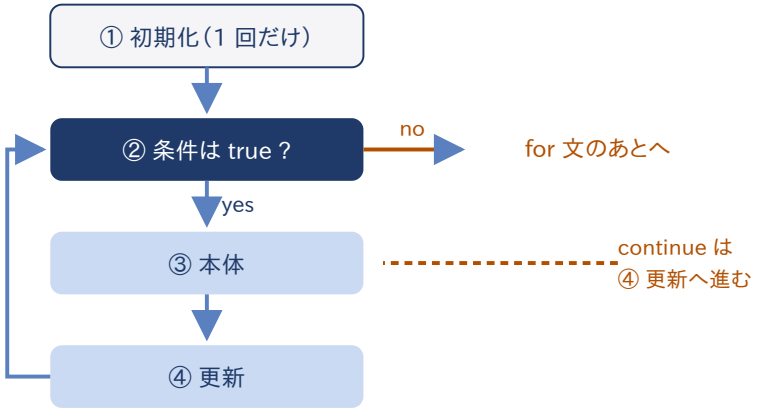
第2章

for 文

初期化は 1 回、更新は本体のあと毎回。

for 文の実行の順番

for (初期化; 条件; 更新) 本体（Claude が作成）



順番を表示で確かめる（Claude が作成）

```
int i = 0;
for (System.out.print("init ");
     i < 2;
     System.out.print("upd ")) {
  System.out.print("body" + i + " ");
  i++;
}
```

```
init body0 upd body1 upd
```

最後の本体のあとも更新は実行される。そのあと条件で false になって終わる。だから「for の前で宣言した i」は、終わったとき条件が false になった値（例では 2）。

出典：JLS 14.14.1.1 Initialization of for Statement・14.14.1.2 Iteration of for Statement（If execution of the Statement completes normally, then … the ForUpdate … are evaluated in sequence from left to right … Second, another for iteration step is performed）／結果は JDK 17 で確かめたもの

初期化部と更新部に書けるもの

for のかっこ	結果（JDK 17）
`int i = 0, j = 10;`	OK（同じ型を複数）
`int i = 0, long j = 0;`	コンパイルエラー
`i = 0, j = 10;`（どちらも宣言済み）	OK（式文を並べる）
更新部 `i += 3, j -= 3`	OK
初期化部 `System.out.print("x")`	OK（メソッド呼び出しも式文）
`for (;;)`	OK（条件なし ＝ true）
`for (int i = 0; i < 3; )`	OK（更新は本体で）

2 つの変数を同時に動かす（Claude が作成）

```
for (int i = 0, j = 10; i < j; i += 3, j -= 3) {
  System.out.print(i + ":" + j + " ");
}
```

```
0:10 3:7
```

i	j	i < j
0	10	true → 表示
3	7	true → 表示
6	4	false → 終わり

初期化部は「ローカル変数の宣言 1 つ」か「式文のカンマ区切り」のどちらか。宣言と式文をまぜることはできない。

出典：JLS 14.14.1 The basic for Statement（ForInit: StatementExpressionList | LocalVariableDeclaration／ForUpdate: StatementExpressionList）・14.4 Local Variable Declarations・14.8 Expression Statements／結果は JDK 17 で確かめたもの

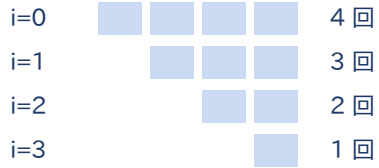
ループ変数のスコープと入れ子のループ

コード	結果（JDK 17）
`for (int i = 0; …) { }` のあと `println(i)`	cannot find symbol
`int i; for (i = 0; i < 3; i++) { }` のあと `println(i)`	3
`int i = 0;` のあと `for (int i = 0; …)`	variable i is already defined
for を 2 つ続け、どちらも `int i`	OK（スコープが別）

スコープ：初期化部で宣言した変数は、条件・更新・本体の中だけで使える。ループのあとで値を使いたいなら、for の前で宣言する。

内側は外側の 1 回ごとに最初から（Claude が作成）

```
int count = 0;
for (int i = 0; i < 4; i++)
  for (int j = i; j < 4; j++)
    count++;
System.out.println(count);   // 10
```



合計 4+3+2+1 = 10

{ } がなくても、内側の for 文全体が外側の本体（1 文）になる。

出典：JLS 6.3 Scope of a Declaration（The scope of a local variable declared in the ForInit part of a basic for statement includes … the Expression and ForUpdate parts …, and the contained Statement）・6.4（It is a compile-time error if the name of a local variable v is used to declare a new variable within the scope of v）／結果は JDK 17 で確かめたもの

第3章

拡張 for 文

配列か Iterable を、先頭から 1 つずつ。

拡張 for のしくみと使える型

配列の拡張 for は、次の基本の for と同じ意味（JLS 14.14.2 の説明を Claude が書き直した）

```
for (int x : a) { … }
// ↓ と同じ
for (int #i = 0; #i < a.length; #i++) {
  int x = a[#i];      // 要素のコピー
  …
}
```

#i はプログラムからは見えない名前。List なら iterator() の hasNext()・next() で同じことをする。

> **ループ変数は毎回新しく宣言される。**だから final や var を付けられる。代入しても元の配列やリストは変わらない（#04）。

: の右	結果（JDK 17）
配列 int[]	OK
List<String>・ArrayList<Integer>	OK（Iterable）
String	for-each not applicable to expression type
StringBuilder・int	for-each not applicable
s.toCharArray()	OK（char[]）

: の左	結果（JDK 17）
var x・final int x	OK
int n（List<Integer> から）	OK（アンボクシング）
宣言済みの x だけ	bad initializer for for-loop

出典：JLS 14.14.2 The enhanced for statement（The type of the Expression must be an array type or a subtype of the raw type Iterable, or a compile-time error occurs）／Java SE 17 API java.lang.Iterable（Implementing this interface allows an object to be the target of the enhanced for statement）／結果は JDK 17 で確かめたもの

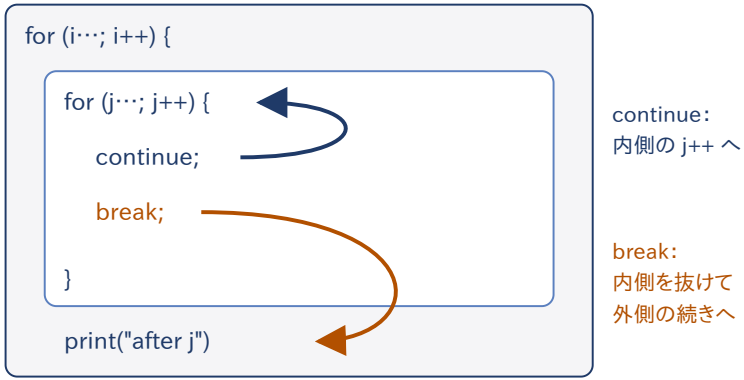
第4章

break・continue・ラベル

ラベルなしはいちばん内側。外へ行くならラベル。

break と continue の行き先

飛ぶ先（Claude が作成）




```java
for (int i = 0; i < 5; i++) {
  if (i == 1) continue;   // 1 を飛ばす
  if (i == 3) break;      // 3 で抜ける
  System.out.print(i + " ");
}
```

```
0 2
```

こんなとき	結果（JDK 17）
入れ子の内側で `break;`	内側だけ抜け、外側は続く
ループの中の switch で `break;`	switch だけ抜ける
while で更新が continue の後ろ	無限ループになりうる
break; の直後の文	unreachable statement

出典：JLS 14.15 The break Statement（A break statement with no label attempts to transfer control to the innermost enclosing switch, while, do, or for statement）・14.16 The continue Statement（the continue target … immediately ends the current iteration and begins a new one）／結果は JDK 17 で確かめたもの

ラベル付き break と continue

```
outer:
for (int i = 0; i < 3; i++) {
  for (int j = 0; j < 3; j++) {
    if (j == 1) continue outer;
    if (i == 2) break outer;
    System.out.print(i + "" + j + " ");
  }
}
```

```
00 10
```

i	j	起きること
0	0	00 を表示
0	1	continue outer → i++ へ
1	0	10 を表示
1	1	continue outer → i++ へ
2	0	break outer → 全部を抜ける

コード	結果（JDK 17）
`loop: if (true) { … break loop; }`	OK（if を抜ける）
`loop: if (true) { continue loop; }`	not a loop label: loop
ループの中の switch で `break a;`（a は for）	for ごと抜ける

> **break ラベル：**ラベルの付いた文を抜ける。その文はループでなくてもよい。
> **continue ラベル：**ラベルの付いたループの次の回へ。ラベルはループ（while・do・for）に付いていなければならない。

Java に goto はない。ラベルを使うのは break と continue だけ。ラベル名は変数名と同じでもよい。

出典：JLS 14.7 Labeled Statements（Unlike C and C++, the Java programming language has no goto statement）・14.15（In this case, the break target need not be a switch, while, do, or for statement）・14.16（the continue target must be a while, do, or for statement, or a compile-time error occurs）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
ループの表示結果	変数の値を 1 回ずつ表に書く。条件で比べる値（i++ か ++i か）と、ループ後の値を分けて考える
while と do-while の差	最初から false なら while は 0 回、do-while は 1 回。do-while の最後の ;
字下げした 2 行	{ } がなければ 1 文目だけがループの中。無限ループの選択肢を疑う
コンパイルできるか	到達できない文（定数の false・true、break の直後）、for の宣言は 1 つの型、ループ変数のスコープ、拡張 for の右は配列か Iterable
for の終わったときの値	更新のあとに条件で終わるので、条件が false になった値
break・continue の結果	ラベルなしはいちばん内側（switch の中の break は switch だけ）。ラベル付きはそのラベルの文

出典：このPDFの各ページの資料

まとめ

while と do-while
前判定は 0 回もある。後判定は必ず 1 回。到達不能はエラー

for 文
初期化 1 回 → 条件 → 本体 → 更新。変数は for の中だけ

拡張 for 文
配列か Iterable。左はその場で宣言。変数はコピー

break・continue
内側に効く。ラベルで外側へ。continue のラベルはループだけ

こんなとき	結果
`do { } while (false)`	1 回実行
`while (false) { … }`	コンパイルエラー
for の i をループの後で使う	cannot find symbol
`for (char c : "abc")`	コンパイルエラー
内側のループで `break;`	外側は続く

次は #08 クラスとオブジェクト。フィールド・メソッド・コンストラクタ、オーバーロードと this へ進む。

出典：このPDFの各ページの資料